landing page

explicación, línea por línea

musica/index.html

1. Cabecera y Configuración (`<head>`)

- `<!DOCTYPE html>`: Le avisa al navegador que este documento está escrito en HTML5.
- `<html lang="es" class="scroll-smooth">`: Define el idioma del documento en español y activa el desplazamiento suave (`scroll-smooth`) para que la página navegue fluidamente cuando el usuario hace clic en los enlaces internos.
- `<meta charset="UTF-8">`: Define la codificación de caracteres en UTF-8 para que el sitio reconozca acentos, letras 'ñ' y símbolos especiales.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: Configura la escala inicial para que la página sea completamente adaptable a móviles, tabletas y computadoras (*responsive design*).
- `<title>AURA BEATS - Cyberpunk Sound Experience</title>`: Es el título que se muestra en la pestaña del navegador.
- `<script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>`: Importa la librería Tailwind CSS vía CDN para usar clases de diseño rápido.
- `<link rel="stylesheet" href="[https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css](https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css)">`: Importa la biblioteca FontAwesome para poder mostrar iconos gráficos (como reproductor, pausar, shuffle, etc.).
- `<link rel="preconnect" ...>` y los enlaces de Google Fonts: Conectan e importan las fuentes **Orbitron** (fuente futurista/cyberpunk) y **Plus Jakarta Sans** (fuente legible para textos generales).

2. Configuración Personalizada de Tailwind y CSS Local (`<script>` y `<style>`)

HTML

```
<script>
    tailwind.config = {
        theme: {
            extend: {
                colors: {
                    music: {
                        bg: '#080810',
```

```
                card: '#121220',
                neonPurple: '#8a2be2',
                neonCyan: '#00f3ff',
                neonPink: '#ff007f'
            }
        },
        fontFamily: {
            cyber: ['Orbitron', 'sans-serif'],
            sans: ['Plus Jakarta Sans', 'sans-serif']
        }
    }
  }
</script>
```

- **`tailwind.config = { ... }`**: Extiende Tailwind con nuestra paleta personalizada:
    - Fondo oscuro (`bg`), tarjetas (`card`) y acentos de color neón (`neonPurple`, `neonCyan`, `neonPink`).
    - Asigna la tipografía `cyber` a Orbitron y `sans` a Plus Jakarta Sans.

CSS
```
.neon-text-cyan {
   text-shadow: 0 0 10px rgba(0, 243, 255, 0.7), 0 0 20px rgba(0, 243, 255, 0.5), 0 0 40px rgba(0, 243, 255, 0.3);
}
.neon-text-pink {
   text-shadow: 0 0 10px rgba(255, 0, 127, 0.7), 0 0 20px rgba(255, 0, 127, 0.5), 0 0 40px rgba(255, 0, 127, 0.3);
}
```

- Aplica resplandores luminosos de texto (*text-shadow*) estilo neón en cyan y rosa.

CSS
```
.neon-box-purple {
   box-shadow: 0 0 20px rgba(138, 43, 226, 0.4), inset 0 0 15px rgba(138, 43, 226, 0.2);
}
```

- Crea una sombra morada tanto exterior como interior (*inset*) alrededor del reproductor de audio.

CSS
```
.cyber-border {
   border: 1px solid rgba(0, 243, 255, 0.3);
   position: relative;
```

```
}
.cyber-border::before {
    content: '';
    position: absolute;
    top: -2px; left: -2px; width: 10px; height: 10px;
    border-top: 2px solid #00f3ff; border-left: 2px solid #00f3ff;
}
.cyber-border::after {
    content: '';
    position: absolute;
    bottom: -2px; right: -2px; width: 10px; height: 10px;
    border-bottom: 2px solid #ff007f; border-right: 2px solid #ff007f;
}
```

- Crea un borde estilo "HUD/Cyberpunk" dibujando esquinas marcadas en la parte superior izquierda (cyan) e inferior derecha (pink) de las tarjetas usando los pseudo-elementos `::before` y `::after`.

CSS

```
.eq-bar {
    animation: bounce 1.2s ease-in-out infinite alternate;
}
@keyframes bounce {
    0% { height: 15%; }
    100% { height: 95%; }
}
```

- **`.eq-bar`**: Asigna la animación `bounce` a cada barra del ecualizador.
- **`@keyframes bounce`**: Define el movimiento continuo haciendo crecer las barras desde el 15% hasta el 95% de su altura.

3. Cuerpo de la Página (`<body>`) y Capas de Luces Ambientales

HTML

```
<body class="bg-music-bg text-white font-cyber min-h-screen relative overflow-x-hidden selection:bg-cyan-500 selection:text-black">

    <div class="absolute top-10 left-1/4 w-96 h-96 bg-music-neonPurple/20 rounded-full blur-3xl pointer-events-none"></div>
    <div class="absolute top-1/3 right-10 w-96 h-96 bg-music-neonCyan/20 rounded-full blur-3xl pointer-events-none"></div>
```

- Configura el fondo de la página con `#080810`, el texto en blanco y la fuente Orbitron.

- Los dos `<div>` de luz ambiental usan gradientes circulares difuminados (`blur-3xl`) de color morado y cyan para crear destellos neón flotantes. `pointer-events-none` evita que interfieran con los clics.

4. Barra de Navegación (`<nav>`)

HTML
```
<nav class="border-b border-gray-800 px-6 py-4 flex justify-between items-center max-w-7xl mx-auto">
  <div class="text-2xl font-black tracking-widest text-transparent bg-clip-text bg-gradient-to-r from-music-neonCyan to-music-neonPink">
    AURA<span class="text-white">//</span>BEATS
  </div>
  <div class="hidden md:flex space-x-8 text-xs tracking-wider text-gray-400 font-sans">
    <a href="#music-lineup" class="hover:text-music-neonCyan transition-colors">LINEUP 2026</a>
    <a href="#music-app" class="hover:text-music-neonCyan transition-colors">AURA STUDIO APP</a>
    <a href="#music-tickets" class="hover:text-music-neonCyan transition-colors">VIP PASSES</a>
  </div>
  <button class="bg-transparent border border-music-neonCyan text-music-neonCyan hover:bg-music-neonCyan hover:text-black px-5 py-2 text-xs font-bold tracking-widest transition-all duration-300">
    GET APP
  </button>
</nav>
```

- **Logo (`AURA//BEATS`)**: Utiliza `bg-clip-text` con un degradado de cyan a rosa para darle un acabado brillante.
- **Menú de enlaces**: En dispositivos pequeños permanece oculto (`hidden`) y en pantallas medianas o superiores se despliega en horizontal (`md:flex`).
- **Boton "GET APP"**: Botón transparente con bordes de luz cyan que cambia al pasar el ratón (*hover*).

5. Sección Principal de Presentación (`Hero Section`)

HTML
```
<section class="max-w-7xl mx-auto px-6 py-16 md:py-24 grid grid-cols-1 lg:grid-cols-2 gap-12 items-center">
```

- Divide la pantalla en dos columnas iguales (`lg:grid-cols-2`) para pantallas grandes: la izquierda contiene los textos y botones principales, mientras que la derecha alberga el reproductor interactivo.

Lado Izquierdo (Textos y Acciones):

- `inline-flex ...`: Una etiqueta superior redondeada que dice "NEXT-GEN SOUND EXPERIENCE".
- `<h1>`: Título principal donde la palabra **SONIDO** destaca con un texto degradado y sombra de neón.
- `<p>`: Texto descriptivo en tipografía limpia (`font-sans`).
- `<a>`: Botón de acción destacado "CONSEGUIR ENTRADAS" que dirige al área de venta de tickets (`#music-tickets`).
- `<button onclick="toggleAudioSim()">`: Botón interactivo que invoca la función Javascript `toggleAudioSim()` para pausar o reanudar el reproductor.

Lado Derecho (Simulador de Reproductor & Ecualizador):

HTML

```
<div class="h-40 bg-black/50 rounded-xl p-4 flex items-end justify-between gap-1 border border-gray-800 mb-6">
    <div class="eq-bar w-full bg-gradient-to-t from-music-neonPurple to-music-neonCyan rounded-t" style="animation-delay: 0.1s"></div>
    ...
    <div class="eq-bar w-full bg-gradient-to-t from-music-neonPurple to-music-neonCyan rounded-t" style="animation-delay: 0.8s"></div>
</div>
```

- Contiene 8 barras verticalmente alineadas. El atributo `style="animation-delay: X.Xs"` aplica un desfase temporal a la animación de cada barra para crear la sensación de un movimiento de espectro de sonido real.
- Debajo se ubican la información del track (`Cybernetic Horizon`) y los botones de control de música (Shuffle, Anterior, Play/Pausa, Siguiente, Repetir).

6. Sección de Artistas (`Lineup Section`)

HTML

```
<section id="music-lineup" class="max-w-7xl mx-auto px-6 py-16 border-t border-gray-800">
    <h2 class="text-2xl md:text-4xl font-bold tracking-wider mb-10 text-center">LINEUP DESTACADO <span class="text-music-neonCyan">//</span> 2026</h2>
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
```

- Dispone un sistema de rejilla adaptable de 3 columnas para tarjetas de artistas.
- Cada tarjeta utiliza la clase CSS `.cyber-border` para incluir los bordes neón en las esquinas.
- Las imágenes incluyen la clase `group-hover:scale-110`, lo que logra que la foto haga un suave zoom al pasar el cursor sobre ella.

7. Sección de Entradas / Captura de Leads (`Ticket Section`)

HTML

```
<section id="music-tickets" class="max-w-4xl mx-auto px-6 py-16 text-center">
```

```
    <div class="bg-gradient-to-b from-music-card to-black p-8 md:p-12 border
border-music-neonPurple rounded-2xl">
        ...
        <div class="flex flex-col sm:flex-row justify-center items-center gap-4">
            <input type="email" placeholder="TU EMAIL CYBERNETICO..." class="bg-black
border border-gray-700 px-5 py-3 text-sm focus:outline-none focus:border-music-neonCyan
w-full sm:w-80 font-sans">
            <button class="w-full sm:w-auto bg-gradient-to-r from-music-neonCyan
to-music-neonPurple text-black font-bold px-8 py-3 text-xs tracking-widest hover:opacity-90
transition-opacity">
                REGISTRARSE
            </button>
        </div>
    </div>
</section>
```

- Bloque central enfocado en convertir al usuario, con un campo `<input>` de correo y un botón llamativo con degradado cyan-morado para suscribirse o reservar entradas.

8. Lógica JavaScript (`<script>`)

HTML

```
<script>
    let isPlaying = true; // 1. Variable de estado para saber si el reproductor está activo o no.

    function toggleAudioSim() {
        isPlaying = !isPlaying; // 2. Invierte el valor actual (de true a false y viceversa).

        // 3. Captura los elementos del DOM que van a cambiar
        const playIcon = document.getElementById('play-icon');
        const mainPlayBtn = document.getElementById('main-play-btn');
        const playText = document.getElementById('play-text');
        const eqBars = document.querySelectorAll('.eq-bar');

        // 4. Evalúa el estado para actualizar interfaz e iconos
        if (isPlaying) {
            playIcon.className = "fa-solid fa-pause text-music-neonCyan";
            mainPlayBtn.className = "fa-solid fa-pause";
            playText.innerText = "PAUSAR SINTETIZADOR";
            // Reanuda la animación CSS de las barras del ecualizador
            eqBars.forEach(bar => bar.style.animationPlayState = 'running');
        } else {
            playIcon.className = "fa-solid fa-play text-music-neonCyan";
            mainPlayBtn.className = "fa-solid fa-play";
            playText.innerText = "REANUDAR SINTETIZADOR";
            // Pausa la animación CSS congelando la altura de las barras
            eqBars.forEach(bar => bar.style.animationPlayState = 'paused');
```

```
    }
  }
</script>
```

JavaScript
<script>

- **Línea 1 (`<script>`):** Es la etiqueta HTML que le avisa al navegador que todo el código que viene a continuación es código en **JavaScript**.

JavaScript
let isPlaying = true;

- **Línea 2 (`let isPlaying = true;`):** Crea una variable llamada `isPlaying` (que significa *"está reproduciendo"*) y le asigna el valor inicial `true` (*verdadero*). Esta variable funciona como un **interruptor o bandera** para saber en todo momento si la música está sonando o si está pausada.

JavaScript
function toggleAudioSim() {

- **Línea 3 (`function toggleAudioSim() {`):** Declara una función llamada `toggleAudioSim` (*alternar simulación de audio*). Esta función contiene todo el grupo de instrucciones que se ejecutará cada vez que el usuario haga clic en los botones de "Reproducir / Pausar".

JavaScript
isPlaying = !isPlaying;

- **Línea 4 (`isPlaying = !isPlaying;`):** Invierte el estado del interruptor. El símbolo `!` significa *"NO"* o *"lo opuesto a"*. Si `isPlaying` era `true`, ahora pasa a ser `false`. Si era `false`, pasa a ser `true`. Es el mecanismo que permite que un mismo botón sirva tanto para pausar como para reanudar.

JavaScript
const playIcon = document.getElementById('play-icon');

- **Línea 5:** Busca en la página HTML el elemento que tenga el ID `'play-icon'` (el icono pequeño del botón secundario) y lo guarda en la variable `playIcon` para poder modificarlo después.

JavaScript
const mainPlayBtn = document.getElementById('main-play-btn');

- **Línea 6:** Busca en la página el elemento con el ID `'main-play-btn'` (el icono del botón circular grande del reproductor) y lo guarda en la variable `mainPlayBtn`.

JavaScript
const playText = document.getElementById('play-text');

- **Línea 7:** Busca en la página el elemento con el ID `'play-text'` (el texto que acompaña al botón) y lo guarda en la variable `playText`.

JavaScript
const eqBars = document.querySelectorAll('.eq-bar');

- **Línea 8:** Busca **todos** los elementos HTML que tengan la clase CSS `.eq-bar` (las barras del ecualizador) y los guarda en una lista llamada `eqBars`.

JavaScript
if (isPlaying) {

- **Línea 9 (`if (isPlaying) {`):** Inicia una condición. Evalúa: *"Si `isPlaying` es `true` (es decir, el reproductor debe estar activo)... ejecútame las siguientes líneas de código"*.

JavaScript
playIcon.className = "fa-solid fa-pause text-music-neonCyan";

- **Línea 10:** Cambia las clases CSS del icono pequeño para que muestre el icono de **pausa** (`fa-pause`) con color cyan neón.

JavaScript
mainPlayBtn.className = "fa-solid fa-pause";

- **Línea 11:** Cambia las clases CSS del botón principal grande para que también muestre el icono de **pausa**.

JavaScript
playText.innerText = "PAUSAR SINTETIZADOR";

- **Línea 12:** Cambia el texto visible en la pantalla por `"PAUSAR SINTETIZADOR"`.

JavaScript
eqBars.forEach(bar => bar.style.animationPlayState = 'running');

- **Línea 13:** Recorre la lista de barras del ecualizador una por una (`forEach`) y le aplica a cada una la propiedad CSS `animationPlayState = 'running'`. Esto hace que las animaciones de subida y bajada de las barras **vuelvan a moverse**.

JavaScript
} else {

- **Línea 14 (`} else {`):** Es la alternativa de la condición. Se ejecuta únicamente cuando `isPlaying` es `false` (es decir, cuando el reproductor debe pausarse).

JavaScript
playIcon.className = "fa-solid fa-play text-music-neonCyan";

- **Línea 15:** Cambia el icono pequeño a la figura de **play / reproducir** (`fa-play`).

JavaScript
mainPlayBtn.className = "fa-solid fa-play";

- **Línea 16:** Cambia el botón principal grande a la figura de **play / reproducir**.

JavaScript
playText.innerText = "REANUDAR SINTETIZADOR";

- **Línea 17:** Cambia el texto visible por `"REANUDAR SINTETIZADOR"`.

JavaScript
eqBars.forEach(bar => bar.style.animationPlayState = 'paused');

- **Línea 18:** Recorre todas las barras del ecualizador y les cambia la animación a `animationPlayState = 'paused'`, lo que las **congela al instante en su posición actual**.

JavaScript
}

- **Línea 19:** Cierra el bloque de la condición `if/else`.

JavaScript
}

- **Línea 20:** Cierra la función `toggleAudioSim`.

JavaScript
</script>

- **Línea 21 (`</script>`):** Etiqueta de cierre que indica el final del bloque de código JavaScript.

---

recetas/index.html

1. Configuración Inicial y Etiquetas de Cabecera (`<head>`)

- **Línea 1 (`<!DOCTYPE html>`):** Define que el documento utiliza el estándar moderno HTML5.
- **Línea 2 (`<html lang="es" class="scroll-smooth">`):** Declara el idioma español y activa el desplazamiento suave (`scroll-smooth`) para los enlaces internos.
- **Línea 3 (`<head>`):** Abre la sección donde se cargan configuraciones, fuentes, estilos y scripts.
- **Línea 4 (`<meta charset="UTF-8">`):** Define la codificación UTF-8 para garantizar que los acentos, la letra 'ñ' y símbolos especiales se desplieguen correctamente.
- **Línea 5 (`<meta name="viewport" content="width=device-width, initial-scale=1.0">`):** Configura el diseño adaptativo (*responsive*) según la pantalla del dispositivo.
- **Línea 6 (`<title>Gourmet Studio...`):** Define el título visible en la pestaña del navegador.
- **Línea 8 (`<script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>`):** Carga la librería de clases CSS de Tailwind vía CDN.
- **Línea 10 (`<link rel="stylesheet" ... font-awesome ...>`):** Carga los íconos de FontAwesome (iconos de reloj, estrella, etc.).
- **Líneas 12–14 (`<link rel="preconnect" ... googleapis ...>`):** Importa desde Google Fonts las tipografías **Playfair Display** (una Serif elegante para títulos) y **Plus Jakarta Sans** (para lectura general).

2. Personalización de Tailwind CSS y Estilos Locales

HTML
  <script>
    tailwind.config = {

```
        theme: {
            extend: {
                colors: {
                    gourmet: {
                        cream: '#FCF8F2',       /* Fondo crema suave */
                        terracotta: '#C85A32',  /* Color primario tipo arcilla/terracota */
                        sage: '#4A5D4E',        /* Verde salvia orgánico */
                        gold: '#D4AF37',        /* Acento dorado */
                        charcoal: '#2B2927'     /* Carbón/Oscuro para textos */
                    }
                },
                fontFamily: {
                    serifGourmet: ['Playfair Display', 'serif'],
                    sans: ['Plus Jakarta Sans', 'sans-serif']
                }
            }
        }
    }
  </script>
```

- **Líneas 16–33:** Configura la paleta de colores personalizada del tema gastronómico (`cream`, `terracotta`, `sage`, `gold`, `charcoal`) y mapea las fuentes CSS para usarlas mediante clases como `font-serifGourmet` y `font-sans`.

HTML

```
  <style>
      .gourmet-shadow {
          box-shadow: 0 20px 40px -15px rgba(200, 90, 50, 0.08);
      }
      .organic-border {
          border-radius: 60% 40% 30% 70% / 60% 30% 70% 40%;
      }
  </style>
</head>
```

- **Líneas 36–38 (`.gourmet-shadow`):** Agrega una sombra suave y difuminada en tono cálido a las tarjetas e imágenes.
- **Líneas 39–41 (`.organic-border`):** Crea una forma asimétrica y orgánica (no perfectamente cuadrada ni redonda) para el marco de la imagen del platillo principal.
- **Línea 42 (`</head>`):** Cierra el encabezado del documento.

3. Cuerpo de la Página (`<body>`) y Navegación (`<nav>`)

- **Línea 43 (`<body class="bg-gourmet-cream text-gourmet-charcoal font-sans min-h-screen">`):** Aplica el fondo crema, el color de texto principal carbón y la fuente por defecto en toda la página.

HTML
```
  <!-- Navigation Bar -->
   <nav class="max-w-7xl mx-auto px-8 py-6 flex justify-between items-center border-b border-stone-200">
       <div class="font-serifGourmet text-2xl md:text-3xl italic font-bold text-gourmet-terracotta">
           Gourmet Studio<span class="text-gourmet-gold">.</span>
       </div>
       <div class="hidden md:flex space-x-10 text-sm tracking-wide text-gourmet-charcoal/80 font-medium">
           <a href="#gourmet-recipes" class="hover:text-gourmet-terracotta transition-colors">Recetas</a>
           <a href="#gourmet-chef" class="hover:text-gourmet-terracotta transition-colors">Chef de Autor</a>
           <a href="#gourmet-philosophy" class="hover:text-gourmet-terracotta transition-colors">Filosofía</a>
       </div>
       <button class="bg-gourmet-sage text-white px-6 py-2.5 rounded-full text-xs font-semibold tracking-wider hover:bg-opacity-90 transition-all shadow-md">
           CLUB GASTRONÓMICO
       </button>
   </nav>
```

- **Línea 46:** Define la barra flotante con ancho máximo centrado (`max-w-7xl mx-auto`) y una línea inferior fina (`border-stone-200`).
- **Líneas 47–49:** El logotipo estilizado en cursiva con la fuente Serif y un punto dorado.
- **Líneas 50–54:** Enlaces del menú superior que se ocultan en pantallas pequeñas y aparecen en escritorios (`hidden md:flex`).
- **Líneas 55–57:** Botón redondeado en verde salvia (`bg-gourmet-sage`) para invitar a unirse al "Club Gastronómico".

4. Sección Principal o Presentación (`Hero Section`)

HTML
```
  <section class="max-w-7xl mx-auto px-8 py-16 md:py-24 grid grid-cols-1 lg:grid-cols-12 gap-12 items-center">
```

- **Línea 61:** Distribuye la sección en una rejilla flexible de 12 columnas (`lg:grid-cols-12`).

Columna Izquierda - Textos y Llamados a la Acción:

- **Líneas 63–65:** Una etiqueta superior pequeña que destaca "Arte Culinario & Alta Cocina".
- **Líneas 66–68:** El título principal `<h1>` combinando texto normal con palabras destacadas en cursiva de color terracotta.
- **Líneas 69–71:** Párrafo descriptivo sobre gastronomía consciente e ingredientes orgánicos.
- **Líneas 73–86:** El botón "EXPLORAR RECETARIO" junto a un bloque de avatars con la métrica de social proof ("+12,000 Miembros Cocinando ahora").

Columna Derecha - Imagen Orgánica e Insignia Flotante:

- **Líneas 89–93:** La imagen del plato gourmet encerrada en la figura con borde orgánico (`organic-border`).
- **Líneas 94–102:** Una tarjeta flotante blanca en la esquina inferior izquierda (`absolute -bottom-6 -left-6`) que presenta la "Receta de la Semana" con detalles como tiempo y nivel de dificultad.

5. Sección de Recetas con Filtros Interactivos

HTML

```
  <section id="gourmet-recipes" class="max-w-7xl mx-auto px-8 py-16 border-t
border-stone-200">
      <div class="text-center max-w-2xl mx-auto mb-12">
          <h2 class="font-serifGourmet text-3xl md:text-5xl font-normal text-gourmet-charcoal
mb-4">Recetario de Autor</h2>
          <p class="text-gray-600 text-sm">Filtra entre nuestras selecciones exclusivas
creadas para despertar tus sentidos.</p>

          <div class="flex flex-wrap justify-center gap-2 mt-8">
              <button onclick="filterRecipes('all')" class="recipe-filter-btn px-5 py-2 rounded-full
text-xs font-medium bg-gourmet-terracotta text-white transition-all">Todas</button>
              <button onclick="filterRecipes('vegetarian')" class="recipe-filter-btn px-5 py-2
rounded-full text-xs font-medium bg-white text-gray-600 hover:bg-stone-200
transition-all">Vegetarianas</button>
              <button onclick="filterRecipes('gourmet')" class="recipe-filter-btn px-5 py-2
rounded-full text-xs font-medium bg-white text-gray-600 hover:bg-stone-200
transition-all">Alta Cocina</button>
              <button onclick="filterRecipes('quick')" class="recipe-filter-btn px-5 py-2
rounded-full text-xs font-medium bg-white text-gray-600 hover:bg-stone-200
transition-all">&amp;lt; 30 Minutos</button>
          </div>
      </div>
```

- **Líneas 106–110:** Encabezado centrado de la sección del recetario.
- **Líneas 111–116:** Botones redondeados para filtrar las recetas. Cada botón ejecuta la función de JavaScript `filterRecipes('categoría')` pasándole como argumento el identificador del filtro (`all`, `vegetarian`, `gourmet`, `quick`).

Grilla de Tarjetas de Recetas:

HTML

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-8" id="recipes-grid">
```

- **Línea 119:** Rejilla que muestra 3 tarjetas por fila en escritorio (`md:grid-cols-3`).
- **Tarjetas 1, 2 y 3 (Líneas 121–168):**
  - Cada tarjeta tiene la clase `.recipe-card` y un atributo de datos especial `data-category="..."` (por ejemplo: `data-category="gourmet"`, `data-category="vegetarian quick"`).
  - Contienen la fotografía con efecto de zoom al pasar el cursor (`group-hover:scale-105`), etiquetas de la categoría, íconos de tiempo de preparación, título y breve descripción.

6. Sección Newsletter y Pie de Página

HTML

```
<!-- Newsletter -->
<section class="bg-gourmet-sage text-white py-16 px-8">
  <div class="max-w-4xl mx-auto text-center space-y-6">
    <h2 class="font-serifGourmet text-3xl md:text-4xl italic">Recibe nuestro menú semanal en tu correo</h2>
    <p class="text-stone-200 text-sm max-w-md mx-auto font-light">Una selección exclusiva de 3 recetas, maridajes de vino y consejos de chefs profesionales.</p>
    <div class="flex flex-col sm:flex-row justify-center gap-3 max-w-md mx-auto">
      <input type="email" placeholder="Tu correo electrónico..." class="px-5 py-3 rounded-full text-gourmet-charcoal text-sm focus:outline-none w-full">
      <button class="bg-gourmet-terracotta text-white px-8 py-3 rounded-full text-xs font-semibold tracking-wider hover:bg-opacity-90 transition-all whitespace-nowrap">
        SUSCRIBIRSE
      </button>
    </div>
  </div>
</section>
```

- **Líneas 172–183:** Bloque con fondo verde salvia que contiene el formulario de suscripción para captar el correo electrónico de los visitantes.
- **Líneas 185–187:** Pie de página (`<footer>`) con derechos de autor y aviso legal.

7. Código JavaScript de Filtrado (`<script>`)

HTML

```
<script>
  function filterRecipes(category) {
    // 1. Selecciona todas las tarjetas de recetas y todos los botones de filtro del HTML
    const cards = document.querySelectorAll('.recipe-card');
    const buttons = document.querySelectorAll('.recipe-filter-btn');
```

```
        // 2. Restablece el estilo de todos los botones a su estado "inactivo" (fondo blanco, texto gris)
        buttons.forEach(btn => {
            btn.className = "recipe-filter-btn px-5 py-2 rounded-full text-xs font-medium bg-white text-gray-600 hover:bg-stone-200 transition-all";
        });
        // 3. Destaca el botón que acaba de ser presionado con el fondo terracotta y texto blanco
        event.target.className = "recipe-filter-btn px-5 py-2 rounded-full text-xs font-medium bg-gourmet-terracotta text-white transition-all";

        // 4. Recorre cada una de las tarjetas para decidir si mostrarla u ocultarla
        cards.forEach(card => {
            const cardCategories = card.getAttribute('data-category'); // Lee la categoría guardada en data-category

            // Si la categoría seleccionada es 'all' o si la tarjeta incluye esa categoría en sus datos:
            if (category === 'all' || cardCategories.includes(category)) {
                card.style.display = 'block'; // Muestra la tarjeta
            } else {
                card.style.display = 'none';  // Oculta la tarjeta
            }
        });
    }
</script>
```

JavaScript
<script>

- **Línea 1:** Abre el bloque de código JavaScript dentro del archivo HTML.

JavaScript
function filterRecipes(category) {

- **Línea 2:** Declara la función llamada `filterRecipes` que recibe un parámetro llamado `category` (por ejemplo: `'all'`, `'vegetarian'`, `'gourmet'`, o `'quick'`). Este valor le indica a la función qué filtro activó el usuario.

JavaScript
// 1. Selecciona todas las tarjetas de recetas y todos los botones de filtro del HTML

- **Línea 3:** Es un comentario explicativo (no se ejecuta).

JavaScript
```
const cards = document.querySelectorAll('.recipe-card');
```

- **Línea 4:** Busca en todo el documento HTML todos los elementos que tengan la clase `.recipe-card` (las tarjetas de recetas) y los guarda dentro de la constante `cards` en forma de lista (NodeList).

JavaScript
```
const buttons = document.querySelectorAll('.recipe-filter-btn');
```

- **Línea 5:** Busca todos los elementos con la clase `.recipe-filter-btn` (los botones de filtro superiores) y los guarda en la constante `buttons`.

JavaScript
```
// 2. Restablece el estilo de todos los botones a su estado "inactivo" (fondo blanco, texto gris)
```

- **Línea 7:** Comentario explicativo.

JavaScript
```
buttons.forEach(btn => {
```

- **Línea 8:** Inicia un bucle `forEach` que va a recorrer uno por uno todos los botones guardados en la lista `buttons`. `btn` representa el botón individual en cada vuelta del ciclo.

JavaScript
```
  btn.className = "recipe-filter-btn px-5 py-2 rounded-full text-xs font-medium bg-white text-gray-600 hover:bg-stone-200 transition-all";
```

- **Línea 9:** Le asigna a cada botón la lista de clases CSS correspondiente al estado **inactivo** (fondo blanco, texto gris y bordes redondeados). Esto "limpia" la selección previa de cualquier otro botón.

JavaScript
```
});
```

- **Línea 10:** Cierra el bucle `forEach` de los botones.

JavaScript
```
// 3. Destaca el botón que acaba de ser presionado con el fondo terracotta y texto blanco
```

- **Línea 11:** Comentario explicativo.

JavaScript
event.target.className = "recipe-filter-btn px-5 py-2 rounded-full text-xs font-medium bg-gourmet-terracotta text-white transition-all";

- **Línea 12:** Utiliza el objeto global `event.target` (el elemento exacto sobre el que el usuario hizo clic) y le cambia sus clases CSS para aplicarle el estado **activo** (fondo color terracota y texto blanco).

JavaScript
// 4. Recorre cada una de las tarjetas para decidir si mostrarla u ocultarla

- **Línea 14:** Comentario explicativo.

JavaScript
cards.forEach(card => {

- **Línea 15:** Inicia otro bucle `forEach` para recorrer cada una de las tarjetas de recetas guardadas en la lista `cards`. En cada iteración, `card` hace referencia a la tarjeta actual.

JavaScript
  const cardCategories = card.getAttribute('data-category'); // Lee la categoría guardada en data-category

- **Línea 16:** Obtiene el valor del atributo personalizado `data-category` que tiene la tarjeta en el HTML (por ejemplo: `"vegetarian quick"`) y lo guarda en la variable `cardCategories`.

JavaScript
  // Si la categoría seleccionada es 'all' o si la tarjeta incluye esa categoría en sus datos:

- **Línea 18:** Comentario explicativo.

JavaScript
  if (category === 'all' || cardCategories.includes(category)) {

- **Línea 19:** Evalúa una condición lógica con dos alternativas (`||` significa **O**):
    1. Si la categoría seleccionada es `'all'` (mostrar todo).
    2. **O** si el texto de `cardCategories` contiene la palabra clave recibida en `category` (por ejemplo, si `"vegetarian quick"` incluye `"vegetarian"`).

JavaScript
```
      card.style.display = 'block'; // Muestra la tarjeta
```

- **Línea 20:** Si la condición se cumple, modifica el estilo CSS directo de la tarjeta asignándole `display = 'block'`, lo que la hace visible en la pantalla.

JavaScript
```
  } else {
```

- **Línea 21:** Indica qué hacer si la condición **no** se cumple (es decir, cuando la tarjeta no coincide con el filtro seleccionado).

JavaScript
```
      card.style.display = 'none';  // Oculta la tarjeta
```

- **Línea 22:** Cambia el estilo CSS directo de la tarjeta a `display = 'none'`, ocultándola por completo del flujo de la página.

JavaScript
```
  }
```

- **Línea 23:** Cierra la estructura condicional `if/else`.

JavaScript
```
});
```

- **Línea 24:** Cierra el bucle `forEach` que recorre las tarjetas.

JavaScript
```
}
```

- **Línea 25:** Cierra el cuerpo de la función `filterRecipes`.

JavaScript
```
</script>
```

- **Línea 26:** Etiqueta de cierre de la sección de código JavaScript.

---

deporte/index.html

1. Configuración de Cabecera y Dependencias (`<head>`)

- **Línea 1 (`<!DOCTYPE html>`):** Indica que el navegador debe interpretar el archivo con el estándar HTML5.
- **Línea 2 (`<html lang="es" class="scroll-smooth">`):** Establece el idioma del sitio en español y activa la navegación con desplazamiento suave.
- **Líneas 3–6:**
  - `<head>`: Contenedor de metadatos.
  - `<meta charset="UTF-8">`: Soporte de caracteres para tildes y símbolos.
  - `<meta name="viewport" ...>`: Ajusta el diseño a pantallas móviles (*Responsive Design*).
  - `<title>PULSEFIT...`: Título que se muestra en la pestaña del navegador.
- **Línea 8 (`<script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>`):** Carga el framework Tailwind CSS desde su CDN.
- **Línea 10 (`<link rel="stylesheet" ... font-awesome ...>`):** Carga la librería de íconos gráficos (como el check `fa-check`).
- **Líneas 12–14:** Importa las fuentes desde Google Fonts: **Teko** (fuente deportiva de impacto) y **Plus Jakarta Sans** (lectura general).

2. Configuración Personalizada de Tailwind y Estilos Inclinados

HTML

```
  <script>
      tailwind.config = {
          theme: {
              extend: {
                  colors: {
                      fit: {
                          yellow: '#CCFF00', /* Amarillo neón deportivo */
                          dark: '#111111',   /* Negro de fondo */
                          cardBg: '#1A1A1A', /* Gris oscuro para tarjetas */
                          orange: '#FF3E00'  /* Naranja de acento */
                      }
                  },
                  fontFamily: {
                      sport: ['Teko', 'sans-serif'],
                      sans: ['Plus Jakarta Sans', 'sans-serif']
                  }
              }
          }
      }
```

```
    </script>
```

- **Líneas 16–33:** Configura la paleta de colores deportivos y mapea las familias tipográficas para usarlas con clases como `font-sport` o `font-sans`.

```
HTML
  <style>
      .skew-x-custom {
          transform: skewX(-12deg);
      }
      .unskew {
          transform: skewX(12deg);
      }
      .btn-skew {
          position: relative;
          transition: all 0.3s ease;
      }
      .btn-skew:hover {
          transform: skewX(-12deg) translateY(-3px);
          box-shadow: 5px 5px 0px #FF3E00;
      }
  </style>
</head>
```

- **Líneas 36–38 (`.skew-x-custom`):** Inclina un elemento -12 grados horizontalmente para darle una estética dinámico/deportiva.
- **Líneas 39–41 (`.unskew`):** Aplica +12 grados a los textos internos para que la caja quede inclinada pero el texto se lea recto.
- **Líneas 42–49 (`.btn-skew`):** Define la animación de los botones en estado *hover* (elevación y sombra naranja).

3. Cuerpo de la Página (`<body>`) y Navegación (`<nav>`)

- **Línea 51 (`<body class="bg-fit-dark text-white font-sport ...">`):** Establece el fondo negro `#111111`, texto blanco y tipografía principal Teko.

```
HTML
  <!-- Navigation Bar -->
   <nav class="max-w-7xl mx-auto px-6 py-5 flex justify-between items-center border-b
border-gray-800">
      <div class="text-3xl font-extrabold tracking-wider text-fit-yellow skew-x-custom italic">
          PULSE<span class="text-white">FIT</span>
      </div>
      <div class="hidden md:flex space-x-8 text-xl tracking-widest text-gray-300
font-semibold">
```

```
        <a href="#fit-metrics" class="hover:text-fit-yellow transition-colors">MÉTRICAS</a>
        <a href="#fit-calc" class="hover:text-fit-yellow transition-colors">CALCULADORA
MACROS</a>
        <a href="#fit-plans" class="hover:text-fit-yellow transition-colors">MEMBRESÍAS</a>
      </div>
      <button class="bg-fit-yellow text-black font-extrabold text-lg px-6 py-1.5 skew-x-custom
btn-skew">
        <span class="block unskew">ÚNETE AHORA</span>
      </button>
    </nav>
```

- **Línea 54:** Contenedor del menú centrado con una línea divisoria inferior oscura (`border-gray-800`).
- **Líneas 55–57:** Logotipo **PULSEFIT** con la 'PULSE' en amarillo neón e inclinada.
- **Líneas 58–62:** Enlaces del menú superior que apuntan a las secciones internas de la página.
- **Líneas 63–65:** Botón "ÚNETE AHORA" inclinado de color amarillo neón con texto corregido (`unskew`).

4. Sección Principal o Presentación (`Hero Section`)

HTML

```
  <section class="max-w-7xl mx-auto px-6 py-12 md:py-20 grid grid-cols-1 lg:grid-cols-2
gap-8 items-center">
```

- **Línea 69:** Rejilla dividida en 2 columnas para computadoras de escritorio (`lg:grid-cols-2`).

Columna Izquierda (Textos y Métricas):

- **Líneas 71–73:** Insignia naranja inclinada con el lema *"SIN EXCUSAS. SOLO RESULTADOS."*.
- **Líneas 74–76:** Título principal con la palabra **POTENCIAL** resaltada en amarillo neón y subrayado naranja.
- **Líneas 77–79:** Párrafo descriptivo en tipografía legible Plus Jakarta Sans (`font-sans`).
- **Líneas 81–88:** Botones de llamados a la acción ("COMENZAR PRUEBA DE 14 DÍAS" y "CALCULAR MI META").
- **Líneas 90–104:** Rejilla de métricas rápidas mostrando estadísticas (98.4% Efectividad, 50K+ Atletas, 24/7 Coaching).

Columna Derecha (Composición de Imagen):

- **Líneas 107–112:** Imagen atlética con filtro en escala de grises que recupera color al pasar el ratón (`hover:grayscale-0`), enmarcada dentro de un contenedor inclinado con borde doble naranja/amarillo.

5. Calculadora Interactiva de Calorías y Macros

HTML

```
  <section id="fit-calc" class="max-w-5xl mx-auto px-6 py-16">
      <div class="bg-fit-cardBg p-8 md:p-10 border-l-8 border-fit-yellow rounded-r-2xl
font-sans">
          ...
          <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
              <div>
                  <label class="block text-xs uppercase font-bold text-gray-400 mb-2">Peso
Actual (kg)</label>
                  <input type="number" id="fit-weight" value="75" class="w-full bg-black border
border-gray-700 px-4 py-3 text-white text-sm focus:border-fit-yellow focus:outline-none">
              </div>
              <div>
                  <label class="block text-xs uppercase font-bold text-gray-400 mb-2">Objetivo
Principal</label>
                  <select id="fit-goal" class="w-full bg-black border border-gray-700 px-4 py-3
text-white text-sm focus:border-fit-yellow focus:outline-none">
                      <option value="1.2">Pérdida de Grasa (Déficit)</option>
                      <option value="1.5" selected>Mantenimiento & Agilidad</option>
                      <option value="1.8">Hipertrofia / Fuerza Pura</option>
                  </select>
              </div>
              <div class="flex flex-col justify-end">
                  <button onclick="calculateCalories()" class="bg-fit-yellow text-black font-black
py-3 px-6 text-sm hover:bg-white transition-colors uppercase">
                      CALCULAR MACROS
                  </button>
              </div>
          </div>

          <div class="mt-6 p-4 bg-black/60 border border-gray-800 flex items-center
justify-between">
              <span class="text-xs text-gray-400 uppercase font-bold">Meta Calórica
Estimada:</span>
              <span id="calorie-result" class="font-sport text-3xl text-fit-yellow font-bold">2,475
KCAL / DÍA</span>
          </div>
      </div>
  </section>
```

- **Línea 116:** Tarjeta con un borde grueso a la izquierda en color amarillo neón (`border-l-8 border-fit-yellow`).
- **Líneas 126–129:** Campo de entrada numérico (`<input id="fit-weight">`) para ingresar el peso en kilogramos.

- **Líneas 130–137:** Menú desplegable (`<select id="fit-goal">`) con factores multiplicadores según el objetivo físico (`1.2` para déficit, `1.5` para mantenimiento, `1.8` para volumen).
- **Línea 139:** Botón que dispara la función JavaScript `calculateCalories()`.
- **Líneas 145–148:** Caja de resultados donde se imprime dinámicamente el valor calculado dentro del span con id `calorie-result`.

6. Tabla de Planes y Membresías

HTML

```
  <section id="fit-plans" class="max-w-7xl mx-auto px-6 py-16 border-t border-gray-800">
      <h2 class="text-4xl md:text-6xl font-black text-center mb-12 tracking-wide">PLANES
DE <span class="text-fit-yellow">ENTRENAMIENTO</span></h2>

      <div class="grid grid-cols-1 md:grid-cols-3 gap-8 font-sans">
```

- **Línea 156:** Crea una rejilla de 3 columnas para los tres niveles de suscripción:
    1. **STARTER PULSE ($29/mes):** Plan inicial con beneficios estándar.
    2. **PRO ATHLETE ($59/mes):** Plan destacado con borde amarillo neón elevado (`md:-translate-y-2`) y etiqueta de *"MÁS POPULAR"*.
    3. **ELITE PERFORMANCE ($99/mes):** Plan prémium con detalles en naranja neón.

7. Código JavaScript de la Calculadora (`<script>`)

Aquí tienes la explicación detallada línea por línea de la función JavaScript:

JavaScript

```
<script>
   // Declara la función que realiza el cálculo al presionar el botón
   function calculateCalories() {

      // 1. Obtiene el valor del campo del peso (id="fit-weight") y lo convierte a número
flotante.
      // Si está vacío o es inválido, toma 75 como valor por defecto.
      const weight = parseFloat(document.getElementById('fit-weight').value) || 75;

      // 2. Obtiene el valor multiplicador seleccionado en el desplegable (id="fit-goal")
      const goalMultiplier = parseFloat(document.getElementById('fit-goal').value);

      // 3. Aplica la fórmula: Peso * 22 * Multiplicador del objetivo, y redondea el resultado
      const baseKcal = Math.round(weight * 22 * goalMultiplier);

      // 4. Selecciona la etiqueta HTML donde se mostrará el resultado
      const resultElem = document.getElementById('calorie-result');

      // 5. Inyecta el resultado con formato numérico (ej: "2,475 KCAL / DÍA")
```

```
        resultElem.innerText = `${baseKcal.toLocaleString()} KCAL / DÍA`;

        // 6. Agrega temporalmente una animación de pulso a la cifra devuelta
        resultElem.classList.add('animate-pulse');

        // 7. Remueve la animación de pulso después de 1 segundo (1000 ms)
        setTimeout(() => {
            resultElem.classList.remove('animate-pulse');
        }, 1000);
    }
</script>
```

HTML
<script>

- **Línea 1:** Abre la etiqueta HTML `<script>`, que le indica al navegador que el contenido a continuación se debe interpretar y ejecutar como código **JavaScript**.

JavaScript
```
    // Declara la función que realiza el cálculo al presionar el botón
```

- **Línea 2:** Es un comentario explicativo. Todo lo que empieza con `//` es ignorado por el navegador y sirve para documentar el código.

JavaScript
```
    function calculateCalories() {
```

- **Línea 3:** Declara la función llamada `calculateCalories` (*calcular calorías*). Esta función agrupa las instrucciones que se ejecutarán cuando el usuario haga clic en el botón *"CALCULAR MACROS"*.

JavaScript
```
        // 1. Obtiene el valor del campo del peso (id="fit-weight") y lo convierte a número
flotante.
        // Si está vacío o es inválido, toma 75 como valor por defecto.
```

- **Líneas 5 y 6:** Comentarios explicativos sobre el primer paso.

JavaScript
```
        const weight = parseFloat(document.getElementById('fit-weight').value) || 75;
```

- **Línea 7:**

    - `document.getElementById('fit-weight')`: Busca en la página el campo de texto de entrada donde el usuario escribe su peso.
    - `.value`: Extrae el texto escrito por el usuario en dicho campo.
    - `parseFloat(...)`: Convierte el texto extraído en un número con decimales (por ejemplo, pasa de `"75.5"` a `75.5`).
    - `|| 75`: Funciona como valor de respaldo (*fallback*). Si el campo está vacío o el usuario ingresó algo que no es un número válido, asigna el valor `75` por defecto.
    - `const weight = ...`: Guarda el resultado numérico en la constante `weight`.

JavaScript
```
// 2. Obtiene el valor multiplicador seleccionado en el desplegable (id="fit-goal")
```

- **Línea 9:** Comentario explicativo del segundo paso.

JavaScript
```
const goalMultiplier = parseFloat(document.getElementById('fit-goal').value);
```

- **Línea 10:**
    - `document.getElementById('fit-goal')`: Busca la lista desplegable `<select>` de objetivos en el HTML.
    - `.value`: Obtiene el valor de la opción seleccionada por el usuario (por ejemplo, `"1.2"`, `"1.5"` o `"1.8"`).
    - `parseFloat(...)`: Convierte ese texto en un número decimal de punto flotante.
    - `const goalMultiplier = ...`: Almacena ese multiplicador en la constante `goalMultiplier`.

JavaScript
```
// 3. Aplica la fórmula: Peso * 22 * Multiplicador del objetivo, y redondea el resultado
```

- **Línea 12:** Comentario explicativo de la fórmula matemática.

JavaScript
```
const baseKcal = Math.round(weight * 22 * goalMultiplier);
```

- **Línea 13:**
    - `weight * 22 * goalMultiplier`: Multiplica el peso por `22` (estimación de metabolismo basal por kg) y luego por el multiplicador del objetivo deportivo seleccionado.
    - `Math.round(...)`: Redondea el resultado matemático al entero más cercano para evitar mostrar decimales extensos.

    - `const baseKcal = ...`: Almacena las calorías finales calculadas en la constante `baseKcal`.

JavaScript
```
// 4. Selecciona la etiqueta HTML donde se mostrará el resultado
```

- **Línea 15:** Comentario explicativo del cuarto paso.

JavaScript
```
const resultElem = document.getElementById('calorie-result');
```

- **Línea 16:** Busca en el documento HTML el elemento `<span>` o `<div>` que tiene el ID `'calorie-result'` (donde se va a publicar la cifra final) y guarda la referencia en la constante `resultElem`.

JavaScript
```
// 5. Inyecta el resultado con formato numérico (ej: "2,475 KCAL / DÍA")
```

- **Línea 18:** Comentario explicativo.

JavaScript
```
resultElem.innerText = `${baseKcal.toLocaleString()} KCAL / DÍA`;
```

- **Línea 19:**
    - `baseKcal.toLocaleString()`: Formatea el número agregando los separadores de miles según el idioma local (por ejemplo, convierte `2475` en `"2,475"` o `"2.475"`).
    - `` `${...} KCAL / DÍA` ``: Utiliza una plantilla de texto (*template literal*) para concatenar el número formateado con el texto `" KCAL / DÍA"`.
    - `resultElem.innerText = ...`: Inyecta la cadena de texto completa dentro de la etiqueta HTML elegida para mostrar el valor en pantalla.

JavaScript
```
// 6. Agrega temporalmente una animación de pulso a la cifra devuelta
```

- **Línea 21:** Comentario explicativo.

JavaScript
```
resultElem.classList.add('animate-pulse');
```

- **Línea 22:** Añade la clase de Tailwind CSS `'animate-pulse'` a la lista de clases del elemento del resultado, provocando que la cifra parpadee suavemente para llamar la atención del usuario.

JavaScript
```
    // 7. Remueve la animación de pulso después de 1 segundo (1000 ms)
```

- **Línea 24:** Comentario explicativo.

JavaScript
```
    setTimeout(() => {
        resultElem.classList.remove('animate-pulse');
    }, 1000);
```

- **Líneas 25–27:**
  - `setTimeout(..., 1000)`: Establece un temporizador que esperará `1000` milisegundos (1 segundo).
  - `() => { ... }`: Una función flecha anónima que se ejecuta automáticamente cuando transcurre ese segundo.
  - `resultElem.classList.remove('animate-pulse')`: Quita la clase de animación para que el número deje de parpadear y vuelva a quedarse estático.

JavaScript
```
}
```

- **Línea 28:** Cierra el bloque de código de la función `calculateCalories`.

HTML
```
</script>
```

- **Línea 29:** Cierra la etiqueta de código JavaScript en el documento HTML.